

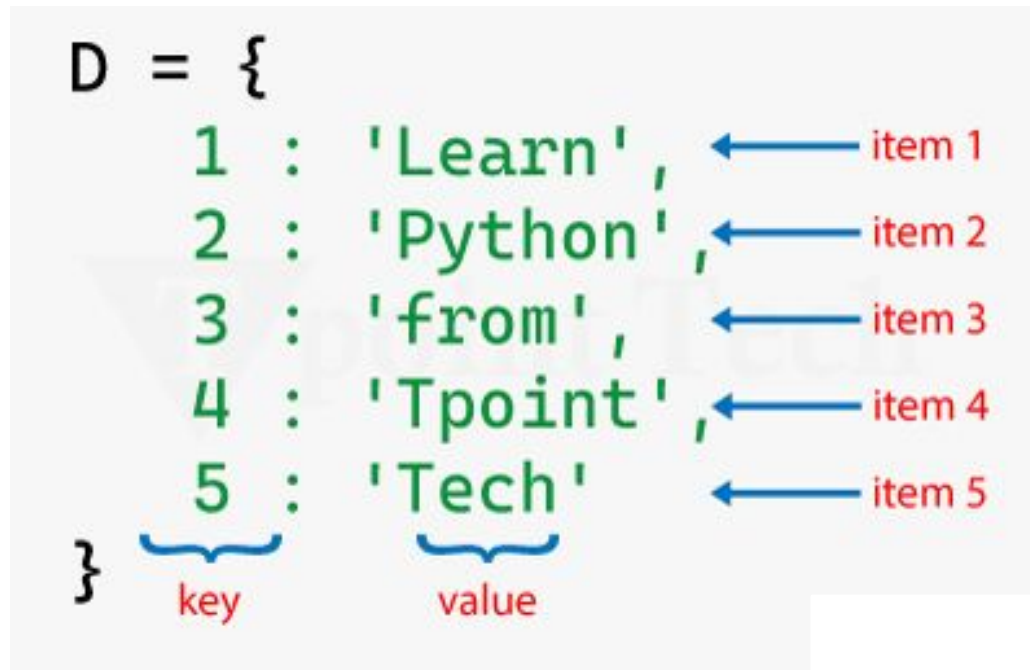
4. DICTIONARY IN PYTHON

PYTHON DICTIONARIES

Used to store data in '**key: value**' pairs.

Characteristics

1. Mutable
2. Ordered(unordered up to 3.7)
3. Indexed
4. Unique keys
5. Heterogeneous



CREATE / DEFINE A DICTIONERIES

- ❑ **empty dictionary**

```
dict_zero = {}
```

- ❑ **using {}**

```
dict_one = {"name": "Lucy", "age": 19, "city": "New Jersey"}
```

- ❑ **using dict()**

```
dict_two = dict(name = "John", age = 21, city = "Havana")
```

ADDING ITEMS TO A DICTIONARY

1. Add a single key–value pair

```
d = {}  
d["name"] = "Athul"  
print(d) #{'name': 'Athul'}
```

2. Add multiple key–value pairs using update()

```
d = {"name": "Athul"}  
d.update({"age": 20, "place": "Kochi"})  
print(d) #{'name': 'Athul', 'age': 20, 'place': 'Kochi'}
```

ADDING ITEMS TO A DICTIONARY

3. Add value to an existing key (overwrite)

```
d = {"marks": 80}
```

```
d["marks"] = 90
```

```
print(d) #{'marks': 90}
```

4. Add value to a key containing a list

```
d = {"fruits": ["apple"]}
```

```
d["fruits"].append("banana")
```

```
print(d) #{'fruits': ['apple', 'banana']}
```

ACCESSING DICT ITEMS

By enclosing that particular key with square brackets '[']'.

Example:

```
d = {"name": "Athul", "age": 20, "place": "Kochi"}
```

```
print(d["name"]) #Athul
```

2. Access a value safely using .get()

(Does NOT give error if key is missing)

```
print(d.get("age"))
```

```
print(d.get("phone", "Not Available"))
```

ACCESSING DICT ITEMS

3.Access nested dictionary

```
student = {  
    "name": "Athul",  
    "marks": {"math": 90, "science": 85}  
}  
print(student["marks"]["science"]) #85
```

CHANGING DICTIONARY ITEMS

1. Change a Value by Key

Just assign a new value:

Example:

```
person = {"name": "sree", "age": 23}
```

```
person["age"] = 25    # change value
```

```
print(person) # {'name': 'sree', 'age': 25}
```

2. Add a New Key-Value or Change Existing

```
person["city"] = "Kochi" # adds new key
```

```
person["name"] = "sreeraj" # updates  
existing key
```

Output: {'name': 'sreeraj', 'age': 25,
'city': 'kochi' }

CHANGING DICTIONARY ITEMS

3. Change Multiple Items at Once Using .update()

Example:

```
person = {"name": "sree", "age": 23}
person.update({"name": "sreeraj", "age": 24})
print(person) #output : {'name': 'sreeraj', 'age': 24}
```

4. Update Inside Nested Dictionaries

Example:

```
students = {
    "sree": {"age": 23, "city": "Kochi"},
    "unni": {"age": 20, "city": "Thrissur"}}
students["sree"]["city"] = "Bangalore"
```

Output : {'sree': {'age': 23, 'city': 'Bangalore'}, 'unni': {'age': 20, 'city': 'Thrissur'}}

CHANGING DICTIONARY ITEMS

5. Rename a Key (Change the dictionary's key)

Note: Python does **not** support direct renaming of keys.

Example:

```
person = {"name": "sree", "age": 23}
```

```
person["username"] = person.pop("name")
```

```
print(person) #output : {'username': 'sree', 'age': 23}
```

REMOVING ITEMS FROM A DICTIONARY

1. Using del (Delete by Key)

Removes a **specific key-value pair**.

Example:

```
person = {"name": "sree", "age": 23, "city":  
"Kochi"}  
  
del person["city"]  
  
print(person) # {'name': 'sree', 'age': 23}
```

2. Using .pop(key)

Removes a key and **returns its value**.

```
person = {"name": "sree", "age": 23}  
  
age = person.pop("age")  
  
print(age)      # 23  
  
print(person)   # {'name': 'sree'}
```

REMOVING ITEMS FROM A DICTIONARY

3. Using .popitem()

Removes the **last inserted** key-value pair.

Example:

```
person = {"name": "sree", "age": 23}
person.popitem()
print(person) #output: {'name': 'sree'}
```

4. Using .clear()

Removes **all items**, leaving an empty dictionary.

Example:

```
person = {"name": "sree", "age": 23}
person.clear()
print(person) #output : {}
```

REMOVING ITEMS FROM A DICTIONARY

Method	Removes	Returns Value	Example
del dict[key]	Specific key	No	delete by key
dict.pop(key)	Specific key	Yes	remove & get value
dict.popitem()	Last item	Yes	LIFO removal
dict.clear()	All items	No	Empty dictionary

DICTIONARY METHODS

Method	Description
<u>get()</u>	This method returns the value associated with a specific key.
<u>update()</u>	This method is utilized to add a new item to the dictionary or update the value of an existing key.
copy()	This method is utilized to return a copy of the dictionary.
<u>pop()</u>	This method removes the item with the given key from the dictionary.
popitem()	This method is utilized to return the last inserted key and value as a tuple.
clear()	This method removes all items from the dictionary.
keys()	This method returns all the keys in the dictionary.
fromkeys()	This method is used to create a new dictionary from a sequence of keys and a value
items()	This method returns a view of the key: value pairs from the dictionary.
<u>setdefault()</u>	This method is used to return value if key exists, otherwise it sets key with default value

DICTIONARY COMPREHENSION

Syntax: **{key_expression : value_expression for item in iterable if condition}**

Example:

```
squares = {x: x*x for x in range(1, 6)} # Output: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

With Conditionals

```
even_squares = {x: x*x for x in range(10) if x % 2 == 0} # Output: {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```